

Target E-commerce Data Analysis (MySQL)



SQL-Based Business Case Study

- Tools : MySQL, SQL
- Database: E-Commerce Dataset (Target)
- GitHub Repository: [target-ecommerce-mysql-analysis](#)

Objective

The objective of this project is to analyze Target Brazil’s e-commerce dataset using MySQL to uncover insights related to customer behavior, order trends, delivery performance, and payment patterns. The analysis focuses on transforming raw transactional data into actionable business insights.

Dataset Overview

Dataset Description

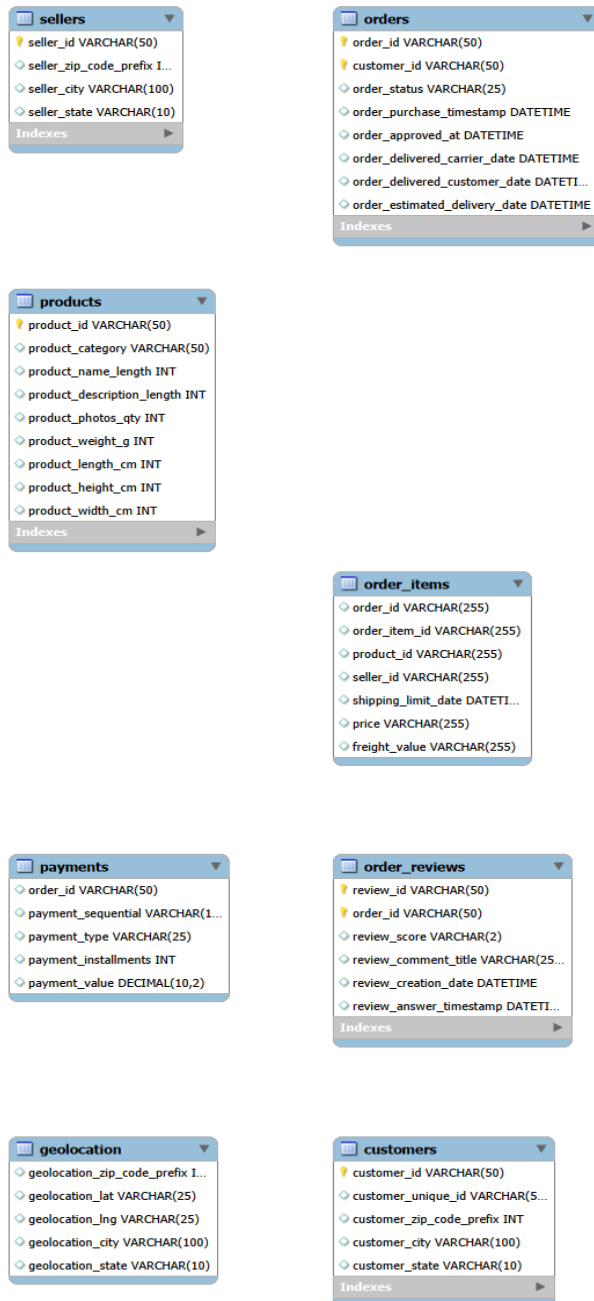
- Total orders: 100k+
- Time Range: Multi-year transactional data
- Granularity: Order-level, item-level, and payment-level

Tables Used

<u>Table Name</u>	<u>Description</u>
customers	Customer demographic details
orders	Order lifecycle and timestamps
order_items	Product price & freight
payments	Payment type and installments
products	Product categories
sellers	Seller information
geolocation	Location mapping

Database Schema

Entity Relationship Diagram



Schema Explanation

- Orders are linked to customers via customer_id
- Order items capture product pricing and freight cost
- Payments capture payment type and installment behavior

Data Loading & Preparation

Data Ingestion

- CSV files loaded using LOAD DATA INFILE
- Date columns converted to proper timestamp formats
- Null and incomplete records filtered during analysis

Key Data Preparation Steps

- Converted multiple date formats into standard DATETIME fields
- Handled corrupted and missing values during ingestion
- Ensured relational consistency across transactional tables

```
-- load csv datafile into created table
/* order_items table has a date + time value column.
When loading non-ISO date formats via LOAD DATA INFILE,
MySQL requires user variables and STR_TO_DATE for proper DATETIME conversion.
*/
LOAD DATA INFILE
'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/order_items.csv'
INTO TABLE order_items
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\r\n'
IGNORE 1 ROWS
(
    order_id,
    order_item_id,
    product_id,
    seller_id,
    @shipping_limit_date, -- Temporary variable. need to use @
    price,
    freight_value
)
SET shipping_limit_date = STR_TO_DATE(@shipping_limit_date,'%d/%m/%Y %H:%i:%s') -- to convert str to date and time format
;
```

Analysis 1: Order Trends

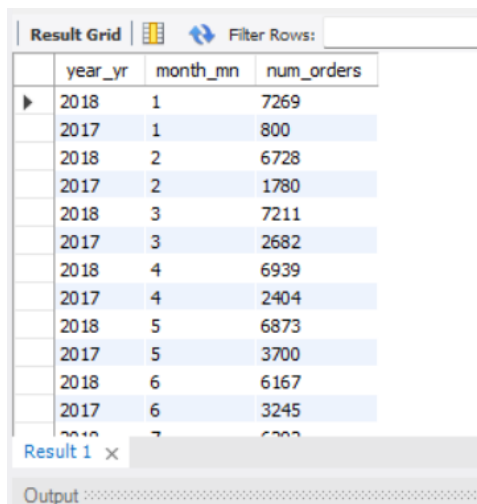
Business Question

-- 1. Is there a growing trend in the no. of orders placed over the past years?

SQL Query

```
SELECT
YEAR(order_purchase_timestamp) AS year_yr,
MONTH(order_purchase_timestamp) AS month_mn,
COUNT(order_id) AS num_orders
FROM orders
GROUP BY 1,2
ORDER BY 2,1 DESC
;
```

Result Snapshot



The screenshot shows a database interface with a 'Result Grid' tab. It displays a table with three columns: 'year_yr', 'month_mn', and 'num_orders'. The data is sorted by month in descending order (1 to 12) and then by year in descending order (2018 to 2017). The table shows a clear upward trend in order volume over time, with 2018 consistently having higher counts than 2017 for every month. There is also a visible seasonal pattern, with higher order counts in the first half of the year (months 1-6) compared to the second half (months 7-12).

year_yr	month_mn	num_orders
2018	1	7269
2017	1	800
2018	2	6728
2017	2	1780
2018	3	7211
2017	3	2682
2018	4	6939
2017	4	2404
2018	5	6873
2017	5	3700
2018	6	6167
2017	6	3245
2018	7	6000
2017	7	3000
2018	8	5800
2017	8	2800
2018	9	5600
2017	9	2600
2018	10	5400
2017	10	2400
2018	11	5200
2017	11	2200
2018	12	5000
2017	12	2000

- Order volumes show a consistent upward trend over time, indicating growing adoption of the e-commerce platform.
- Clear month-over-month seasonality is observed, suggesting predictable demand cycles that can support inventory and capacity planning.

Analysis 2: Regional Insights

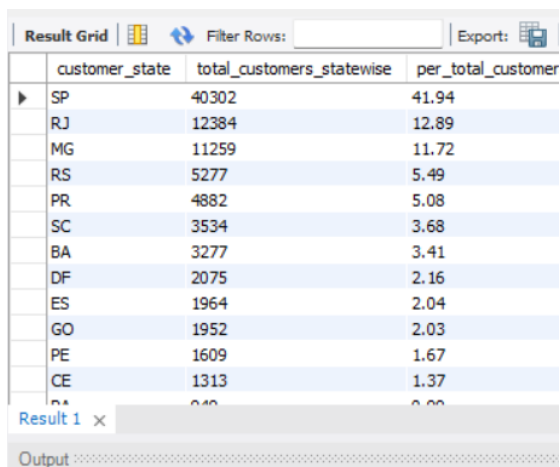
Business Question

Which states contribute the most to overall orders?

SQL Query

```
-- 2. How are the customers distributed across all the states?
SELECT
customer_state,
COUNT(DISTINCT customer_unique_id) AS total_customers_statewise,
ROUND((COUNT(DISTINCT customer_unique_id)/(SELECT COUNT(DISTINCT customer_unique_id) FROM customers) * 100),2) AS per_total_customers
FROM customers
GROUP BY 1
ORDER BY 2 DESC
;
```

Result Snapshot



The screenshot shows a database interface with a 'Result Grid' tab. It displays a table with four columns: 'customer_state', 'total_customers_statewise', and 'per_total_customer'. The data is sorted by 'total_customers_statewise' in descending order. The first row is for 'SP' (Sao Paulo) with 40,302 customers and a 41.94% share. The second row is for 'RJ' (Rio de Janeiro) with 12,384 customers and a 12.89% share. The table continues with other states like MG, RS, PR, SC, BA, DF, ES, GO, PE, and CE, showing a decreasing trend in both customer count and percentage share.

customer_state	total_customers_statewise	per_total_customer
SP	40302	41.94
RJ	12384	12.89
MG	11259	11.72
RS	5277	5.49
PR	4882	5.08
SC	3534	3.68
BA	3277	3.41
DF	2075	2.16
ES	1964	2.04
GO	1952	2.03
PE	1609	1.67
CE	1313	1.37

- Customer demand is highly concentrated in a small number of states, with Sao Paulo (SP) accounting for the highest share of orders.
- A limited set of high-performing states drives a disproportionate percentage of total revenue and order volume, highlighting regional demand imbalance.

Analysis 3: Economic Impact

Business Question

How does revenue vary across regions?

SQL Query

```
SELECT
c.customer_state,
SUM(p.payment_value) AS total_orderprice_statewise,
ROUND(SUM(payment_value)/COUNT(DISTINCT p.order_id),2) AS average_orderprice_statewise
FROM customers c
JOIN orders o
ON o.customer_id = c.customer_id
JOIN payments p
ON p.order_id = o.order_id
GROUP BY 1
ORDER BY 3 DESC
;
```

Result Snapshot

Result Grid			
Filter Rows:		Export:	Wrap Cell Content:
	customer_state	total_orderprice_statewise	average_orderprice_statewise
▶	PB	141545.72	264.08
	AC	19680.62	242.97
	RO	60866.20	240.58
	AP	16262.80	239.16
	AL	96962.06	234.77
	PA	218295.85	223.89
	TO	61485.33	219.59
	PI	108523.97	219.24
	RR	10064.62	218.80
	SE	75246.25	214.99
	RN	102718.13	211.79
	CE	279464.03	209.18
	TX	187320.00	206.21

Result 1 ×

Output

- Average order value varies by state, influenced by customer purchasing power and product mix.

Analysis 4: Delivery Performance

Business Question

Were orders delivered early, on time, or late?

SQL Query

```
SELECT
  order_id,
  DATEDIFF(order_delivered_customer_date,order_purchase_timestamp) AS time_to_deliver,
  DATEDIFF(order_delivered_customer_date,order_estimated_delivery_date) AS diff_estimated_delivery,
  CASE WHEN DATEDIFF(order_delivered_customer_date,order_estimated_delivery_date) < 0 THEN 'Early'
        WHEN DATEDIFF(order_delivered_customer_date,order_estimated_delivery_date) = 0 THEN 'On time'
        ELSE 'Late'
  END AS delivery_status
FROM orders
WHERE order_delivered_customer_date IS NOT NULL      -- Filter out items which are not delivered/canceled/null
;
```

Result Snapshot

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

Fetch rows

	order_id	time_to_deliver	diff_estimated_delivery	delivery_status
▶	00010242fe8c5a6d1ba2dd792cb16214	7	-9	Early
	00018f77f2f0320c557190d7a144bdd3	16	-3	Early
	000229ec398224ef6ca0657da4fc703e	8	-14	Early
	00024acbcd0a6daa1e931b038114c75	6	-6	Early
	00042b26cf59d7ce69dfabb4e55b4fd9	25	-16	Early
	00048cc3ae777c65dbb7d2a0634bc1ea	7	-15	Early
	00054e8431b9d7675808bcb819fb4a32	8	-17	Early
	000576fe39319847cbb9d288c5617fa6	5	-16	Early
	0005a1a1728c9d785b8e2b08b904576c	10	0	On time
	0005f50442cb953dcd1d21e1fb923495	2	-19	Early
	00061f2a7bc09da83e415a52dc8a4af1	5	-11	Early
	00063b381e2406b52ad429470734ebd5	11	0	On time
	0006ec9db01a64e59a68b2c340bf65a7	7	-22	Early
	0008288aa423d2a3f00fcb17cd7d8719	13	-8	Early

Result 1

×

Output

- The majority of orders were delivered on time, indicating effective logistics planning.
- Southern states consistently showed faster delivery time compared to other regions.
- Late deliveries were more common in states with higher freight costs.

Analysis 5: Payment Behavior

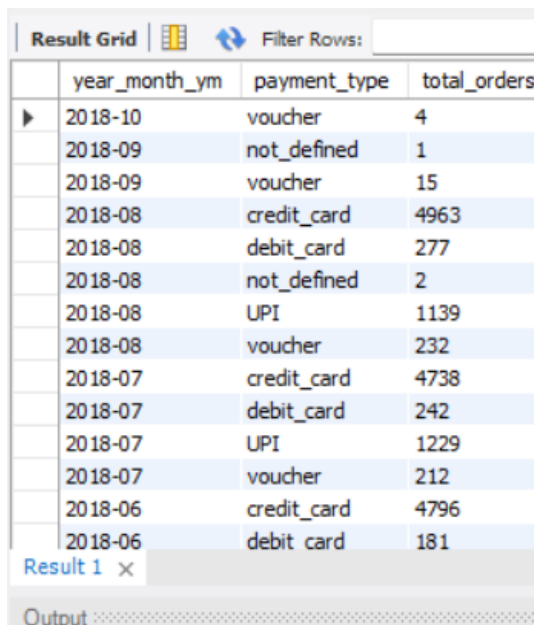
Business Question

What are the dominant payment methods?

SQL Query

```
SELECT
DATE_FORMAT(o.order_purchase_timestamp,'%Y-%m') AS year_month_ym,
payment_type,
COUNT(DISTINCT o.order_id) AS total_orders
FROM payments p
JOIN orders o
ON o.order_id = p.order_id
GROUP BY 1,2
ORDER BY 1 DESC,2 ASC
;
```

Result Snapshot



The screenshot shows a database query result grid with the following columns: year_month_ym, payment_type, and total_orders. The data is sorted by year_month_ym in descending order, and then by total_orders in ascending order. The results show that credit cards are the most common payment method, followed by debit cards, UPI, vouchers, and not_defined.

	year_month_ym	payment_type	total_orders
▶	2018-10	voucher	4
	2018-09	not_defined	1
	2018-09	voucher	15
	2018-08	credit_card	4963
	2018-08	debit_card	277
	2018-08	not_defined	2
	2018-08	UPI	1139
	2018-08	voucher	232
	2018-07	credit_card	4738
	2018-07	debit_card	242
	2018-07	UPI	1229
	2018-07	voucher	212
	2018-06	credit_card	4796
	2018-06	debit card	181

Result 1 x

Output

Insights

- Credit cards dominate as the preferred payment method across all time periods.
- Payment method preferences remain stable over time, indicating consistent customer behavior.

Key Insights Summary

Key Business Insights

- Orders display strong seasonal patterns
- Certain states dominate revenue and order volume
- Delivery performance varies significantly by region
- Credit card payments are the most preferred method
- Freight cost impacts overall order value

Skills Demonstrated

- Advanced SQL (CTEs, Window Functions, DENSE_RANK, LAG)
- Data modeling & relational joins
- Date & time transformations
- Data cleaning during ingestion
- Business problem translation into SQL logic

Conclusion

This project demonstrates the ability to work with large-scale transactional data, design relational schemas, perform advanced SQL analysis, and translate complex outputs into business-ready insights. The findings can support decision-making in logistics optimization, regional demand planning, and payment strategy improvements.